



TALLER PRÁCTICO

Programación Web con Inteligencia Artificial

De una idea conceptual a una aplicación web publicada en producción.

 HTML5 & CSS3

 JavaScript

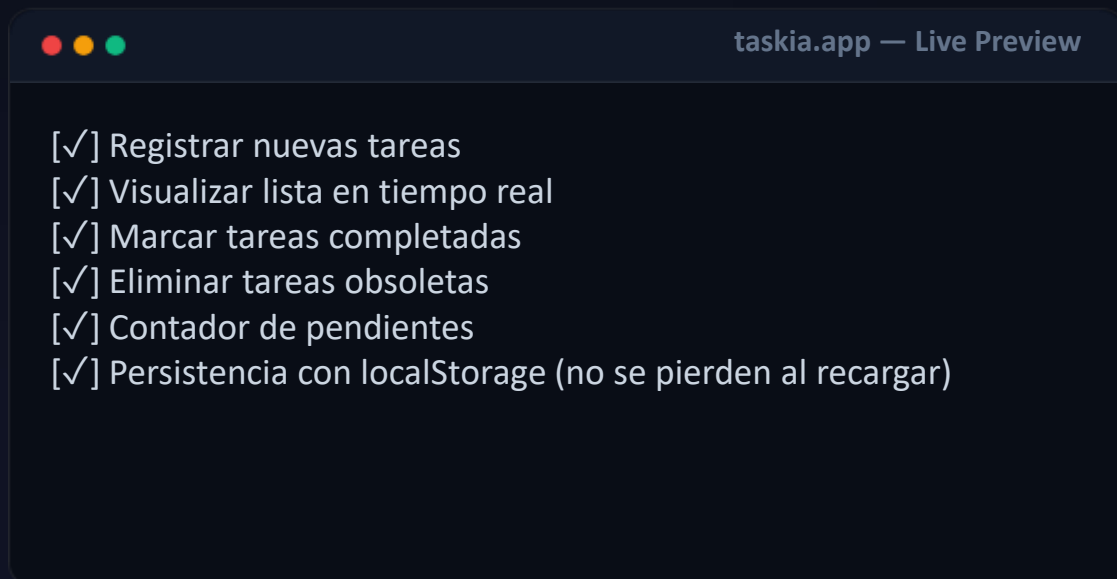
 Git & GitHub

 GitHub Pages

 Asistente IA

 Flujo del taller: Idea → Código → Pruebas → Corrección → Control de versiones → Deploy.

¿Qué vamos a construir? TaskIA



El Producto Final

Una aplicación web moderna, rápida y funcional que no se quedará en tu computadora: quedará desplegada en la nube con acceso público vía **GitHub Pages** para tu portafolio.

Criterio de éxito: Código limpio, comprensible y mantenido por ti.

Objetivos del Taller



Fundamentos Web

Integrar de forma armónica HTML estructural, estilos modernos con CSS y dinamismo interactivo con JavaScript.



IA como Copiloto

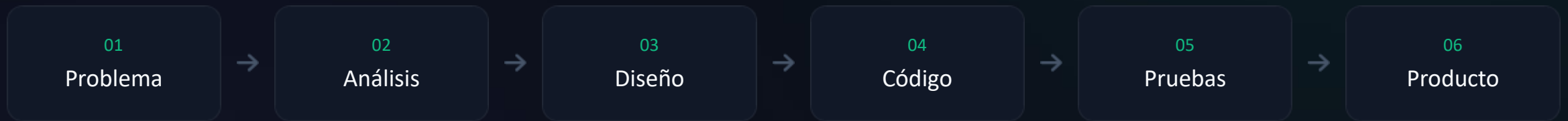
Diseñar prompts técnicos precisos, analizar código generado, depurar errores y evitar alucinaciones del modelo.



Control & Despliegue

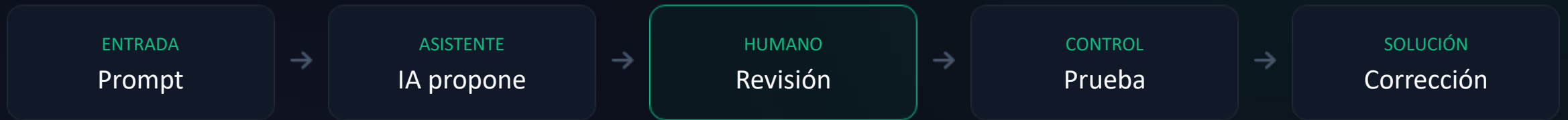
Controlar versiones con Git, sincronizar en GitHub y publicar un sitio estático funcional en GitHub Pages.

Desarrollo Web Tradicional



💡 **Principio clave:** La llegada de la IA no elimina la ingeniería de software ni la lógica fundamental. La potencia y transforma la velocidad de ejecución.

Desarrollo Asistido por IA



«La IA propone soluciones. El programador toma las decisiones arquitectónicas.»

— Axioma de Ingeniería del Taller

Roles de la IA en Desarrollo



Generador & Tutor

Produce estructuras iniciales y explica bloques de sintaxis o APIs que aún no dominas con ejemplos claros.



Debugger & Reviewer

Interpreta stack traces complejos, localiza errores de lógica y audita posibles malas prácticas en tu código.



Refactor & Tests

Simplifica funciones redundantes, mejora nombres de variables y propone casos de prueba de borde (edge cases).

El Riesgo del Copiar y Pegar

- ⚠ Métodos y funciones inventadas que no existen en el estándar.
- ⚠ Dependencias innecesarias que ralentizan tu aplicación.
- ⚠ Introducción silenciosa de vulnerabilidades y bugs de memoria.
- ⚠ Código oscuro que el desarrollador no puede mantener ni explicar.

🛡 Regla de Oro

«Nunca integres a tu repositorio una sola línea de código que no seas capaz de explicar con tus propias palabras.»

Anatomía de una **App Web**



HTML5

Estructura & Semántica: Define los bloques fundamentales, textos, formularios, botones y accesibilidad del documento.



CSS3

Estilo & Presentación: Modela la estética, colores, espaciados, tipografías y el comportamiento responsivo en pantallas.



JavaScript

Comportamiento & Lógica: Responde a eventos del usuario, altera el DOM, procesa datos y sincroniza almacenamiento.

HTML: Estructura y Semántica

¿Qué elementos existen?

HTML no decide colores ni animaciones; responde únicamente a qué entidades componen la interfaz de usuario.

- > Jerarquía de encabezados claros (`<h1>TaskIA</h1>`)
- > Entradas accesibles (`<input type="text">`)
- > Botones de acción semánticos (`<button>Agregar</button>`)

index.html (Estructura base)

```
<main class="app-card">
  <h1>TaskIA</h1>
  <form id="form-tarea">
    <input type="text" id="input-tarea"
      placeholder="¿Qué necesitas hacer?"
      required />
    <button type="submit">Agregar</button>
  </form>
  <ul id="lista-tareas"></ul>
</main>
```

CSS: Diseño y Responsividad

Control visual total

Define la personalidad y la ergonomía de uso de la aplicación web:

 Paleta de color y contraste accesible (WCAG).

 Grillas fluidas y Flexbox moderno.

 Media queries para teléfonos y escritorios.

css/styles.css

```
button {  
    background: #10b981;  
    color: #ffffff;  
    border: none;  
    padding: 10px 20px;  
    border-radius: 8px;  
    font-weight: 600;  
    cursor: pointer;  
    transition: opacity 0.2;  
}  
  
button:hover {  
    opacity: 0.9;  
}
```

JavaScript: **Lógica y Eventos**

El motor dinámico

Convierte una maqueta inerte en una herramienta viva con la que el usuario puede interactuar de manera fluida.

 Escucha eventos en tiempo real (click, submit, keypress).

 Valida entradas y evita envíos vacíos.

 Sincroniza datos en memoria con la pantalla.

js/app.js

```
const form = document.querySelector("#task-form");

form.addEventListener("submit", (event) => {

  event.preventDefault();
  const input = document.querySelector("#task-input");

  if (input.value.trim() !== "") {
    agregarTarea(input.value.trim()); input.value = "";
  }
});
```

Arquitectura del Proyecto **TaskIA**

Árbol de Directorios

```
taskia/
├── index.html    <!-- Estructura semántica -->
├── css/
│   └── styles.css <!-- Presentación y estilos -->
└── js/
    └── app.js     <!-- Lógica y comportamiento -->
```

 **Separación de Responsabilidades:** Cada capa tiene un único motivo de cambio.

 **Rutas relativas:** Permite despliegues sin fricción en subdirectorios como GitHub Pages.

 **Zero Build:** Sin bundlers ni Node.js para entender las bases puras de la web.

Primer Reto: Setup en VS Code



Acción en tu equipo

1. Abre Visual Studio Code.
2. Crea la carpeta principal taskia/.
3. Genera manualmente los tres archivos base.
4. Enlaza el CSS en el y el JS antes de cerrar .

Verificación de enlaces

```
<!-- Dentro de <head> en index.html -->
<link rel="stylesheet" href="css/styles.css">

...

<!-- Justo antes de cerrar </body> en index.html -->
<script src="js/app.js"></script>
```

¿Qué es un Prompt?

Un prompt no es una búsqueda en Google. Es la especificación técnica que guía a una red neuronal.

En ingeniería de software: **A mayor contexto y restricciones, menor tasa de alucinación.**

Prompt ingenuo: "Haz una app de tareas".

→ Resultado impredecible, frameworks ajenos, código incomprensible.



La clave: Especificación

Trata a la IA como un programador junior altamente veloz: necesita saber exactamente el qué, el cómo y qué tiene estrictamente prohibido hacer.

Estructura del Prompt Técnico

01 · ROL & CONTEXTO

Quién eres y qué hay

Define el estado del proyecto y el perfil del modelo: "Actúa como desarrollador frontend senior en Vanilla JS".

02 · REQUISITOS

Qué debe incluir

Especifica elementos exactos: inputs, IDs específicos, clases semánticas y flujo esperado.

03 · RESTRICCIONES

Qué NO debe hacer

"No uses React ni Bootstrap. Solo Vanilla JS modular. Responde únicamente con el bloque de código solicitado".

Prompt: Generando index.html

Prompt para Copilot / ChatGPT / Claude

[CONTEXTO]: Estoy construyendo una app web llamada TaskIA en JS y CSS puro.

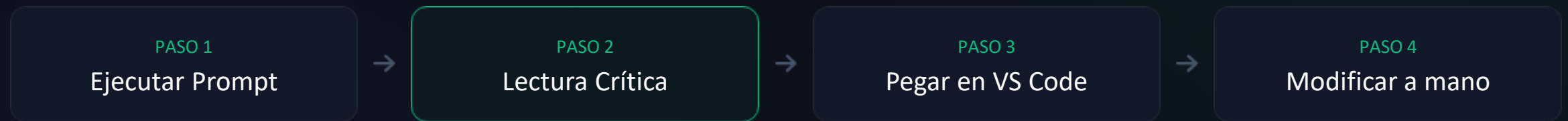
[OBJETIVO]: Crea la estructura de index.html con accesibilidad y diseño limpio.

[REQUISITOS]: - Título h1 con icono. - Formulario con input de texto y botón 'Agregar Tarea'. - Contenedor . - Contador de tareas pendientes y botón de limpiar completadas.

[RESTRICCIONES]: No uses librerías externas ni CSS en línea. Vincula css/styles.css y js/app.js.

[SALIDA]: Entrega únicamente el código HTML completo.

Práctica: Generar y Auditar HTML



 **Pregunta de auditoría:** ¿Qué etiquetas reconozco de inmediato? ¿Hay algún elemento sobrante o mal estructurado que deba limpiar?

El Desarrollo Iterativo

 No busques la perfección en la primera respuesta.

 Construye capa sobre capa: primero esqueleto, luego estilos, luego lógica.

 Retroalimenta a la IA señalando con exactitud qué ajustar.

Bucle de Refinamiento

Prompt inicial



Resultado preliminar



Evaluación humana en el navegador



Prompt de corrección específico



Solución robusta y validada

Práctica: Estilizado con CSS

Prompt para Estilos

Genera css/styles.css para el HTML previo:

- Tema oscuro contemporáneo (#0f172a).
- Contenedor centrado con max-width de 540px.
- Botones con microinteracción hover.
- Tipografía moderna del sistema.
- 100% responsivo para pantallas móviles.
- Sin librerías externas

Refinamientos específicos

Aprende a pedir microajustes:



"Reduce el padding vertical en móviles".



"Aumenta la jerarquía visual del botón principal".



"Añade una línea tachada cuando la tarea esté completada".

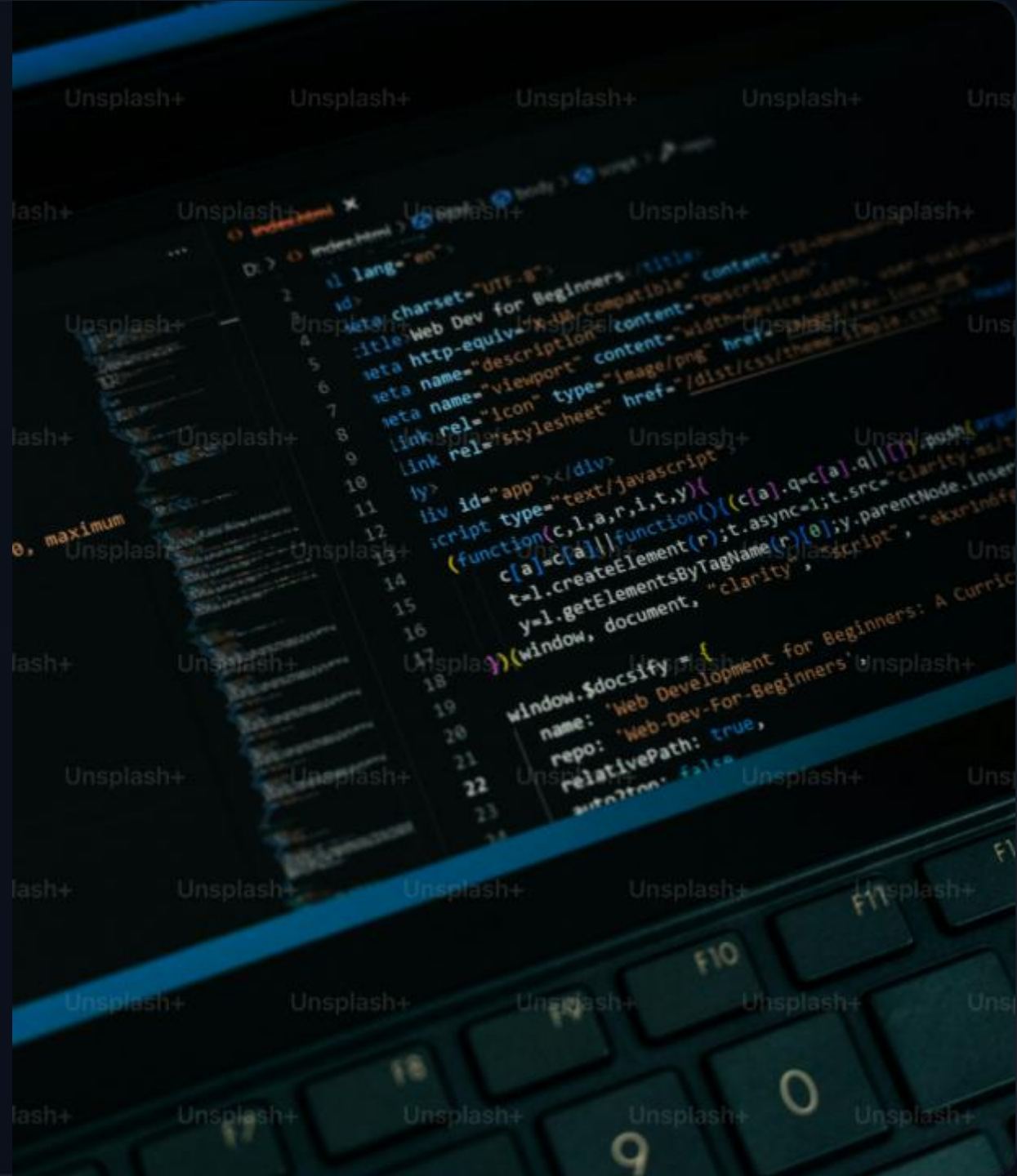
Del Diseño al Comportamiento

HITO 2 COMPLETADO

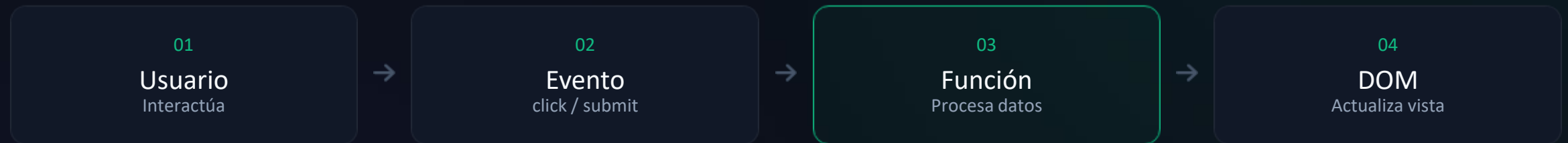
Ya contamos con el esqueleto semántico (HTML) y la apariencia estética visual (CSS).

Nuestra aplicación se ve profesional, pero aún es estática. Es momento de dotarla de vida y memoria mediante JavaScript.

⚡ Objetivo: Conectar las acciones del usuario con el estado del sistema.



El Modelo de **Eventos Web**



Capturando el evento submit

```
formulario.addEventListener("submit", (e) => { e.preventDefault(); // Previene que la página se recargue agregarTarea(input.value); });
```

Funciones Atómicas en TaskIA



Mutación

agregarTarea()
eliminarTarea(id)
completarTarea(id)



Presentación

renderizarTareas()
actualizarContador()



Persistencia

guardarTareas()
cargarTareas()

Estrategia: Prompts Incrementales

✗ Antipatrón

«Genera todo el JavaScript completo para mi aplicación con todas las funciones de golpe.»

→ Provoca código monolítico, difícil de depurar y propenso a omitir casos límite.

✓ Práctica Recomendada

«Implementa únicamente la función agregarTarea() vinculada a este HTML existente.»

→ Validas, pruebas en navegador y sólo entonces pides la función de eliminar.

El DOM: El Árbol Vivo

¿Cómo ve JavaScript la web?

El DOM es una representación estructurada en objetos de cada elemento HTML.

 `document.querySelector('#id')`

 `document.createElement('li')`

 `elemento.classList.toggle('done')`

Manipulación directa del DOM

```
// 1. Localizar el elemento en pantalla const lista =  
document.querySelector("#task-list");  
// 2. Crear un nuevo nodo const item =  
document.createElement("li"); item.textContent =  
"Aprender Git";  
// 3. Insertarlo en el documento lista.appendChild(item);
```

El Estado de la **Aplicación**

La verdad de los datos

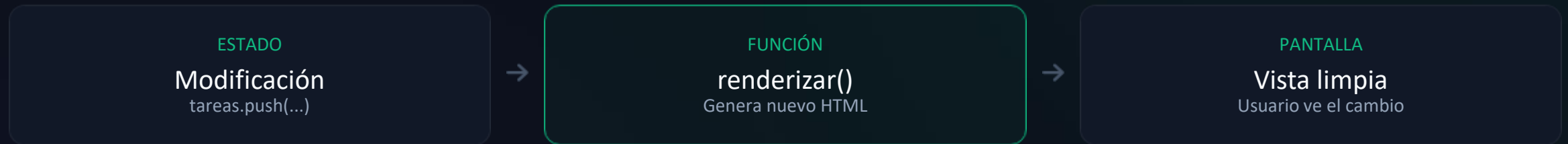
No guardes la información dentro del HTML. Mantén un arreglo de objetos en JavaScript que represente el estado actual.

Regla moderna: La interfaz de usuario es una proyección directa del estado en memoria.

Estructura del Estado en app.js

```
let tareas = [ { id: 1718000001, texto: "Estudiar selectores CSS", completada: true }, {  
id: 1718000002, texto: "Crear repositorio en GitHub", completada: false } ];
```

El Ciclo de **Renderizado**



Función de renderizado declarativo

```
function renderizarTareas() { const lista = document.querySelector("#task-list"); lista.innerHTML = ""; // Limpia la lista previa tareas.forEach((t) => { lista.innerHTML += ` ${t.texto} ✕ `; }); }
```

Persistencia: **localStorage**

¿Qué pasa al recargar?

Sin persistencia, la memoria RAM del navegador se reinicia y las tareas desaparecen.

localStorage almacena cadenas de texto en el navegador del usuario de forma permanente.

JSON: El puente entre objetos de JavaScript y texto plano.

Guardar y Recuperar

```
// Guardar tareas serializadas
function guardar() { localStorage.setItem("taskia_data",
JSON.stringify(tareas)); }
// Recuperar al iniciar la aplicación function cargar() { const datos =
localStorage.getItem("taskia_data"); tareas = datos ?
JSON.parse(datos) : []; renderizarTareas(); }
```

Depurar también es Programar

Lo que hace el novato

Borra todo el archivo cuando algo falla y le pide desesperado a la IA que lo vuelva a escribir de cero.

→ No aprende la causa y acumula nuevos errores invisibles.

Lo que hace el ingeniero

Reproduce el error sistemáticamente, abre la consola, lee la traza e inspecciona qué variable vale null.

→ Corrige la causa raíz con precisión quirúrgica.

La Consola: Tu Instrumento F12

 **Apertura rápida:** Presiona [F12] o [Ctrl + Shift + I].

 **Uncaught TypeError:** Intentaste leer propiedades de algo inexistente.

 **Línea y archivo:** Te indica exactamente dónde ocurrió la excepción.

Console - DevTools

```
> Uncaught TypeError: Cannot read properties of null (reading 'addEventListener') at app.js:14:8 [Diagnóstico]: El selector document.querySelector("#task-form") devolvió null porque el script se ejecutó antes de que el HTML existiera en el DOM.
```

La IA como **Debugger** Estratégico

Prompt para Diagnóstico

Tengo este error en la consola de Chrome: 'Uncaught TypeError: Cannot read properties of null' Este es mi bloque en app.js: [pegar líneas 10 a 20] Por favor: 1. Explica qué significa este error específico. 2. Identifica las dos causas más probables. 3. Dime qué revisar en mi HTML sin reescribir todo mi archivo.



Aprende en cada error

Al pedir explicaciones causales en lugar de parches ciegos, conviertes cada bug en una lección de arquitectura web.

¿Qué es Git?

La máquina del tiempo del software

Git es un sistema de control de versiones distribuido que registra cada hito y cambio en tu código fuente.

Adiós a carpetas como proyecto_final_v2_esto_si.zip

🕒 Posibilidad de volver atrás en caso de romper algo.

🔗 Registro histórico de decisiones técnicas.

Línea de tiempo de commits

```
* 4a8f9c1 (HEAD -> main) feat: agregar persistencia localStorage * 2b7e12d  
feat: renderizado dinámico de tareas * 89a0f44 feat: diseño responsivo con CSS  
* 13c5a90 chore: estructura inicial del proyecto
```

El Flujo Básico de Git



Comandos de Terminal en VS Code

```
git init
# Inicia el rastreador en taskia/
git status # Observa archivos modificados
git add .
# Prepara todos los cambios
git commit -m "feat: interfaz base y lógica de TaskIA"
# Guarda la instantánea
```

Diferencia Crítica: **Git** vs **GitHub**



Git (El Motor Local)

Herramienta que se ejecuta en tu computadora sin conexión a internet. Rastrea deltas de código y ramas.



GitHub (La Nube)

Plataforma web en la nube que hospeda tus repositorios Git, facilita la colaboración y ofrece automatizaciones como CI/CD y Pages.

Subiendo el Proyecto a **GitHub**

El puente hacia la nube

1. Crea un nuevo repositorio público en github.com con el nombre **taskia**.
2. Conecta tu repositorio local con la URL remota mediante estos tres comandos estándar:

Terminal

```
# 1. Vincular origen remoto
git remote add origin https://github.com/tu-usuario/taskia.git
# 2. Asegurar rama principal
git branch -M main
# 3. Empujar código a la nube
git push -u origin main
```

¿Qué es **GitHub Pages**?



Hosting Gratuito

Convierte directamente los archivos estáticos de tu repositorio en un sitio web con certificado SSL (HTTPS) gratuito.



Despliegue Continuo

Cada vez que haces un nuevo git push a la rama main, tu sitio web se actualiza automáticamente.



Ideal para Portafolio

URL pública para compartir con reclutadores, clientes y colegas de forma inmediata.

Límites de GitHub Pages

❌ No Ejecuta Servidor

GitHub Pages **NO** interpreta lenguajes de backend ni bases de datos tradicionales:

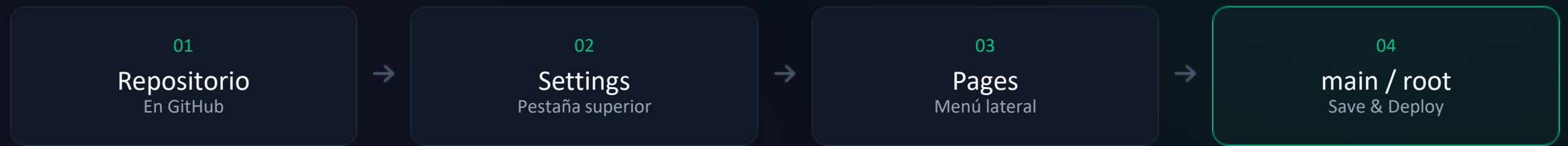
- ✗ Node.js / Express
- ✗ PHP / Laravel
- ✗ Python / Django
- ✗ MySQL / PostgreSQL

✅ 100% Client-Side

Ejecuta a la perfección tecnologías que el navegador interpreta por sí mismo:

- ✓ HTML5 semántico
- ✓ CSS3 moderno
- ✓ JavaScript del cliente
- ✓ Almacenamiento localStorage

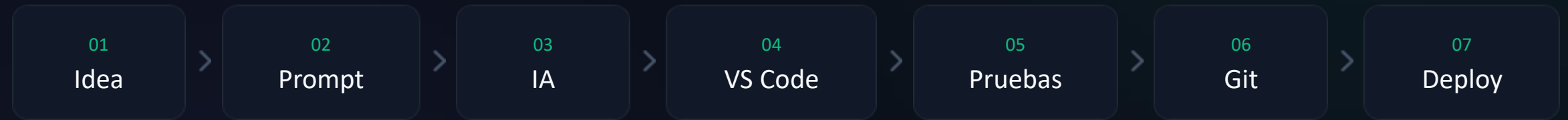
Publicando TaskIA



¡Aplicación en línea!

URL pública activa: <https://.github.io/taskia/>

El Ciclo de Ingeniería Completo



Has recorrido de punta a punta el flujo profesional de un desarrollador moderno asistido por inteligencia artificial.

Reto: Evolucionar TaskIA

Escoge una nueva funcionalidad:

 Modo Oscuro / Claro con botón toggle.

 Selector de prioridades (Alta, Media, Baja).

 Buscador en tiempo real para filtrar tareas.

 Edición en línea del texto de la tarea.

Condición del Reto

Prohibido pedir:

«Vuelve a generar toda la aplicación con la nueva función».

Debes instruir a la IA para modificar únicamente la función o clase pertinente conservando tu código actual.

Conclusión: El Nuevo Desarrollador

«Programar con Inteligencia Artificial no significa delegar el pensamiento; significa amplificar tu capacidad de crear reteniendo la responsabilidad del código.»

— Taller: Programación Web con IA

Analizar mejor

Formular problemas con rigor arquitectónico antes de escribir código.

Crear más rápido

Acelerar la creación de prototipos e implementaciones repetitivas.

Aprender siempre

Desentrañar el código propuesto para convertirlo en conocimiento propio.

Preguntas para el Cierre

¿Qué rol asumió la IA y cuál asumiste tú?

¿Por qué no podemos confiar ciegamente en la primera respuesta?

¿Cuál es la diferencia operativa entre Git y GitHub?

¿Por qué GitHub Pages no soporta PHP ni bases de datos?

¿Qué utilidad tiene la consola DevTools al depurar?

¿Qué mejora arquitectónica le harías a TaskIA mañana?

Checklist de **Resultado Final**

- ✓ **Aplicación funcional:** TaskIA opera localmente sin errores en la consola.
- ✓ **Triada web limpia:** Código modular en index.html, styles.css y app.js.
- ✓ **Persistencia de datos:** Las tareas se conservan en localStorage tras recargar.
- ✓ **Historial en Git:** Repositorio versionado con commits descriptivos.
- ✓ **Publicación en producción:** Sitio web accesible públicamente en GitHub Pages.